
Align2Act: Instruction-Tuned Models for Human-Aligned Autonomous Driving

Kanishkha Jaisankar
Masters in Data Science
New York University
kj2675@nyu.edu

Sunidhi Tandel
Masters in Computer Engineering
New York University
sdt9243@nyu.edu

Team [11]

Abstract

Motion planning in complex scenarios is a core challenge in autonomous driving. Conventional methods apply predefined rules or learn from driving data to generate future trajectories. Recent approaches leverage the reasoning capabilities of large language models (LLMs) for decision-making in driving scenarios. Despite promising results, it remains unclear whether LLMs truly capture the underlying human logic of driving. In this paper, we propose **Align2Act**, a motion planning framework that transforms instruction-tuned LLMs into interpretable planners aligned with human behavior. We derive driving instructions based on human reasoning patterns (e.g., anticipate hazards, yield near intersections) and traffic rules (e.g., stop at red lights, stay within lane boundaries). We then employ an **Align2ActChain** module to guide step-by-step reasoning, producing both the rationale and the trajectory. **Align2ActDriver** supports both instruction injection and learning from large-scale driving data, enhancing interpretability and behavioral alignment. Different from prior works focused on synthetic or open-loop settings, we evaluate our method on the real-world nuPlan closed-loop benchmark (Test 14-random), demonstrating improved planning quality and human-likeness. **Our code is publicly available at** : <https://github.com/Jkanishkha0305/Align2Act>.

1 Introduction

Motion planning in autonomous driving is a core challenge in Embodied AI, requiring agents to perceive, reason, and act in dynamic environments [1]. Unlike traditional software systems, autonomous vehicles must process complex multi-agent interactions, predict future states, and execute safe behaviors while adapting to unpredictable road conditions [2]. Traditional motion planning methods are typically categorized into rule-based approaches, which rely on predefined heuristics such as optimization-based trajectory planning or finite-state machines, and learning-based models, which leverage deep neural networks for decision-making [3, 1]. While rule-based methods provide interpretability, they struggle with adaptability, whereas learning-based models exhibit flexibility but often lack robustness and causal reasoning, particularly in unseen scenarios. Both approaches face difficulties in handling rare edge cases, necessitating a more cognitively grounded alternative [4].

Recent research suggests that Large Language Models (LLMs) could offer a novel perspective by incorporating knowledge synthesis, causal reasoning, and multi-step planning [5]. LLMs can integrate traffic rules, human-like heuristics, and adaptive reasoning into a single framework [6], but their direct application to motion planning remains uncertain. Key challenges include whether LLMs possess an embodied understanding beyond static reasoning [7], their ability to replicate human-like decision-making from structured driving data [8], and ensuring that their predictions

remain interpretable, safe, and aligned with real-world constraints [9]. Since LLMs lack built-in priors for physical interactions, their decisions are prone to hallucinations in real-world settings, making structured alignment mechanisms necessary to enforce safety constraints while maintaining adaptability [10]. The integration of explicit reasoning steps within the motion planning pipeline is critical, where planning should be decomposed into sub-tasks such as collision avoidance, rule compliance, and situational awareness. This instruction-driven approach enhances interpretability and ensures safer, more transparent decision-making in complex environments [11].

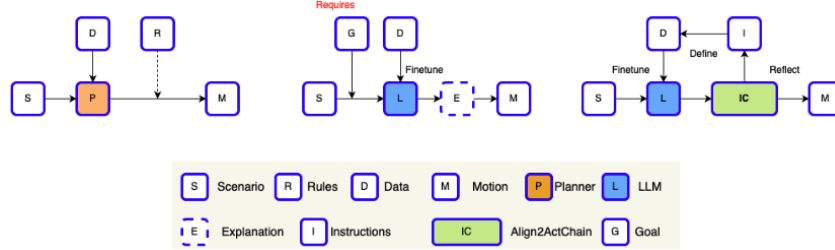


Figure 1: The Align2Act framework: **(D)**ata and **(S)**cenario inputs are processed through **(R)**ules and **(L)**LM reasoning to generate **(M)**otion plans via the **(IC)** Align2ActChain. **(P)**lanner and **(E)**xplanation modules ensure interpretability, while **(I)**nstructions and **(G)**oal alignment refine human-like behavior.

2 Related Work

We reviewed prior work in three areas to position our approach against existing benchmarks and gaps.

LLM-Based Planning Agents. LLM agents have demonstrated strong planning capabilities across simulated and physical domains. For example, Voyager was the first LLM-powered agent to autonomously explore and complete the Overworld tech tree in Minecraft, showcasing long-term planning and skill chaining [12]. LLMs have also enabled the simulation of a virtual town with 25 memory-augmented agents capable of reflection, planning and coordination [13]. They have facilitated multi-agent collaboration through modular frameworks [14] and have been fine-tuned for semantically grounded robotic control in real-world settings [15]. These advances make LLMs promising for high-level planning in autonomous driving.

Motion Planning in Autonomous Driving without LLMs. Before LLMs, motion planners were primarily rule-based, learning-based, or hybrid. Rule-based models like IDM use heuristics for safe vehicle following [16]. Learning-based approaches such as End-to-End driving [17], Conditional Imitation Learning [18] and UrbanDriver [19] learn driving strategies from real-world data. PlanTF leverages Transformers to generalize under long-tail scenarios [20]. Hybrid methods like PDM integrate rule-based logic with learning-based models for improved robustness [21]. However, these systems lack the reasoning flexibility and semantic richness that LLMs offer.

LLM-Based Motion Planning in Autonomous Driving. Recent work integrates LLMs directly into the planning pipeline. LanguageMPC introduces cognitive reasoning pathways and converts high-level decisions into executable commands [22]. GPT-Driver formulates planning as a language modeling problem using linguistic tokens for trajectory prediction [23]. Driving-with-LLMs fuses vectorized scene inputs with pretrained LLMs to improve contextual understanding [24]. LMDrive [4] pioneers end-to-end closed-loop control with LLMs and LLM-ASSIST [25] enhances closed-loop planners with language-based reasoning. These methods offer improved adaptability, interpretability and safety in complex driving settings.

3 Proposed Approach

3.1 Motion Planning as Language Modeling

We formulate motion planning as a conditional language generation problem. Given a structured textual input describing the vehicle’s context and planning goals, a large language model is used to generate both the final driving trajectory and the corresponding reasoning steps. The goal is to produce human-aligned, interpretable plans that are safe and context-aware.

Let T denote the desired trajectory, and X_t be the planning context at time step t . This input includes environmental observations O , the ego vehicle’s current state S_t , motion system metadata S_y , and planning instructions I :

$$X_t = \{O, S_t, S_y, I\} \quad (1)$$

This formulation enables the planner to reason over a combination of physical state and semantic instruction. Rather than predicting a trajectory directly from numerical features, we use language to encode the spatial-temporal context, system constraints, and behavioral intent.

The output Y_t consists of two components: the generated trajectory T and a structured reasoning trace, referred to as the **Align2ActChain**, denoted by I_c . This intermediate chain articulates the model’s step-by-step logic in human-readable terms, providing interpretability and post-hoc traceability:

$$Y_t = \{I_c, T\} \quad (2)$$

To generate this output, we pass the input X_t through a transformation function F , which produces log-probabilities over the model vocabulary. These log-probabilities quantify the model’s confidence in the next-token prediction during decoding:

$$Y_{\log} = F(X_t) \quad (3)$$

This probabilistic output is then post-processed using a decoding mechanism that balances randomness and confidence. Specifically, temperature scaling $T(\cdot, t)$ adjusts the sharpness of the distribution, followed by the Softmax function S to obtain probabilities, and finally top- p sampling P to select high-likelihood completions:

$$Y_t = P(S(T(Y_{\log}, t)), p) \quad (4)$$

Here, the temperature parameter t modulates output diversity—higher values encourage exploration, while lower values enforce determinism. The top- p parameter defines a cumulative probability threshold for sampling, ensuring only the most plausible outputs are considered. This decoding process generates structured plans that balance safety, feasibility, and alignment with human-like decision-making.

3.2 Instruction-Based Behavior Alignment

To teach the model human-aligned behavior, we employ imitation learning via prompt-based supervision. The input X_t is decomposed into two parts: the instruction context I_t and the scene-based planning input I_p :

$$X_t = \{I_t, I_p\} \quad (5)$$

The instruction context I_t includes a description of the vehicle’s motion system S_y and the desired planning behavior I :

$$I_t = \{S_y, I\} \quad (6)$$

The motion system description S_y encodes vehicle-specific properties such as heading convention, yaw orientation, vehicle dimensions (modeled as quadrilaterals), and dynamic parameters like velocity and acceleration. The planning instructions I define task-level goals, such as "turn right" or "decelerate to yield," enabling the model to reason semantically rather than only kinematically.

The planning input I_p contains the ego-centric scene representation, composed of environment observations O and the ego vehicle’s state S_t :

$$I_p = \{O, S_t\} \quad (7)$$

Observations O include object categories (vehicles, cyclists, pedestrians), their relative positions, motion predictions, and semantic map elements such as lane lines, traffic lights, and speed limits. The self-state S_t includes historical trajectory information and the vehicle’s current kinematic state, which together help the model plan smooth transitions.

3.3 Align2ActChain: Structured Reasoning for Interpretability

The output of our model includes both the trajectory T and the Align2ActChain I_c , which is a structured explanation of the decision-making process:

$$Y_t = \{I_c, T\} \quad (8)$$

The **Align2ActChain** decomposes the planning process into four stages that reflect typical human reasoning in driving scenarios.

Preliminary Planning: This step identifies a high-level maneuver such as continuing along the current lane, initiating a lane change, or preparing for a turn. It provides the initial semantic intent based on spatial context and scenario geometry.

Collision Prediction: The planner forecasts the future positions of surrounding agents and evaluates their proximity to the ego vehicle. Agents within 3 meters are flagged as potential hazards, while those within 1.5 meters are considered critical, requiring immediate action to ensure safety.

Traffic Context Assessment: This stage incorporates external constraints such as traffic light states, speed limits, and lane boundary conditions. Red signals prompt a full stop, yellow signals indicate caution, and green signals allow movement. Speed regulations are enforced by suppressing acceleration near or above the limit. Lane boundary violations are also considered during maneuver selection.

Final Action Integration: The model synthesizes all prior reasoning to determine a safe and contextually appropriate driving action. This final decision is then mapped to a continuous trajectory T , which is executed in simulation for downstream evaluation.

Trajectory T is represented as a sequence of poses over time, each with x , y , and θ (yaw) coordinates:

$$T = \{(x_1, y_1, \theta_1), (x_2, y_2, \theta_2), \dots, (x_n, y_n, \theta_n)\} \quad (9)$$

This structured output is compatible with nuPlan’s simulation framework and can be executed via trajectory tracking controllers for open-loop and closed-loop evaluation.

3.4 Align2ActDriver: Model Architecture and Fine-Tuning

The Align2ActDriver framework uses LLaMA-2-7B as the backbone model and is fine-tuned using Low-Rank Adaptation (LoRA) [26], which enables efficient parameter updates with minimal memory overhead. The input sequence X_t is first passed through a preprocessing function P , followed by an embedding layer E that transforms it into high-dimensional vector space:

$$X = E(P(X_t)) \quad (10)$$

This embedding X is then fed into the model F_m , which includes attention modules and normalization. The output is a vocabulary-sized logit vector representing next-token probabilities:

$$Y_{\log} = F_m(X, \text{Att}(X, A, B, b), wR(X)) \quad (11)$$

In this formulation, $\text{Att}(X, A, B, b)$ refers to the attention mechanism, augmented by LoRA matrices A and B , which act as low-rank approximations that reduce the number of trainable parameters. The bias term b and the RMSNorm function with weight w help stabilize training and improve

convergence. Only LoRA-specific parameters (A, B, b, w) are updated during training, while the base LLaMA weights remain frozen, ensuring computational efficiency and performance.

This architecture enables scalable fine-tuning of large models for faster adaptation to the motion planning domain without requiring full retraining. The result is a planner that generates interpretable, human-aligned driving behavior based on high-level textual instructions and scene context.

4 Experiments

4.1 Dataset and Preprocessing

We conduct our experiments using the publicly available **nuPlan** dataset, which provides a large-scale, high-fidelity set of autonomous driving scenarios collected in diverse urban environments. The dataset comprises complex real-world driving data, including diverse traffic participants, signalized intersections, varying road geometries, and dynamic objects such as pedestrians and cyclists.

Each scenario in nuPlan is represented over a temporal window, containing ego vehicle states, surrounding agents, map information, and sensor-derived features. Following the methodology of PlanTF [27], we utilized feature builders to extract the past 2 seconds of ego vehicle motion and predict the next 8 seconds, sampled at a temporal resolution of 0.1 seconds. However, to balance computational efficiency with predictive accuracy, our experiments used a coarser resolution of 0.5 seconds during training and inference.

To ensure coordinate invariance and facilitate generalization, all positional data are transformed such that the ego vehicle is positioned at the origin $(0, 0)$ of the coordinate frame with zero yaw. All surrounding agents’ positions and yaw angles are converted relative to the ego-centric frame. A maximum of 32 surrounding agents are included per scenario, prioritized by spatial proximity.

In addition to numerical features, our approach incorporates structured language-based instructions. For instruction fine-tuning, we construct natural language prompts that describe the scenario context, intended maneuvers, and constraints. These instructions are encoded and concatenated with trajectory and state information to condition the LLM. The final dataset includes approximately one million labeled instruction-driven scenarios curated from the nuPlan Mini Split.

4.2 Experiment Setup

We fine-tuned the **LLaMA-2-7B** language model using **LoRA (Low-Rank Adaptation)**[26] to efficiently adapt it for interpretable motion planning. We selected **LLaMA-2-7B** over other comparable models (e.g., Mistral-7B, Falcon-7B) due to its open-source availability and seamless compatibility with **LoRA** fine-tuning on resource-constrained hardware such as the NVIDIA T4 GPU. The base weights, tokenizer, and configuration files (including `params.json`) were sourced from **Hugging-Face** after receiving approval from Meta. Training was conducted on a single NVIDIA T4 GPU with 8 CPU cores using the Lightning AI Studio platform. Training spanned approximately 12 hours over **6 epochs**, including a warm-up phase during the first epoch to stabilize training dynamics.

The LoRA configuration employed a rank of 16, with tuning applied to both the **bias** and **normalization layers**. This setup limited the number of trainable parameters to approximately **41.6 million**, significantly reducing the computational overhead compared to full fine-tuning. The model was trained using bf16 precision to optimize GPU memory usage without sacrificing numerical stability.

We utilized the `llama-2-accessory` toolkit for training, which includes utilities for LoRA, tokenizer integration, and low-rank updates. The tokenizer was based on **SentencePiece**, with a vocabulary of 32,000 tokens and a maximum input sequence length of 512 tokens. For generation tasks, we allowed input prompts of up to 12,288 tokens to accommodate detailed instruction planning.

The learning rate was initialized at 5×10^{-5} , decaying to a minimum value of 5×10^{-6} , with a weight decay factor of 0.02 to prevent overfitting. We used gradient clipping at a value of 2 to stabilize training and accumulated gradients over 2 steps to simulate a larger batch size, as the batch size was constrained to 1 due to GPU limitations.

During inference, generation was performed with a temperature of 0 for deterministic decoding and a top- p sampling threshold of 0.75 to balance exploration and certainty. Attention computations

were accelerated using **FlashAttention** to reduce memory consumption, while **APEX** enabled mixed-precision training with improved throughput.

Our training pipeline used data parallelism via Scaled Dot Product Attention (SDP) and was optimized for low-memory, single-GPU execution. This configuration enabled scalable fine-tuning and inference, making the Align2ActDriver framework suitable for efficient motion planning experimentation.

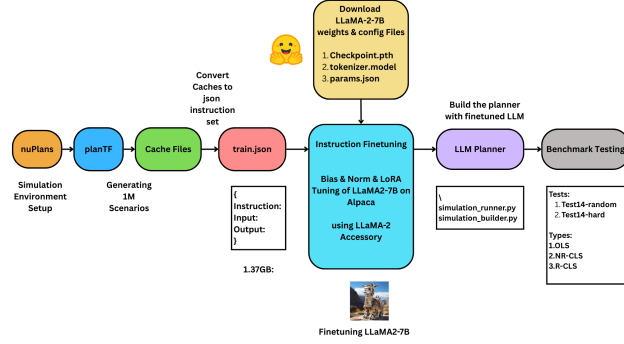


Figure 2: End-to-end project workflow: from data acquisition and simulation setup to training, fine-tuning, and benchmarking with LLM-based planners.

4.3 Evaluation Metrics

To evaluate the quality, safety, and human-likeness of the generated driving plans, we adopt the official evaluation protocol provided by the nuPlan benchmark. This framework includes both open-loop and closed-loop metrics, designed to assess spatial accuracy, trajectory feasibility, rule compliance, and interaction with dynamic agents.

Open-Loop Score (OLS) measures the planner’s ability to imitate human trajectories without simulation feedback. It aggregates multiple sub-metrics, including the *Average Distance Error (ADE)*, which computes the mean Euclidean distance between predicted and ground-truth trajectories over the planning horizon, and the *Final Distance Error (FDE)*, which captures deviation at the final predicted timestep. Additionally, orientation accuracy is evaluated using *Average Heading Error (AHE)* and *Final Heading Error (FHE)*. The *Missing Rate* accounts for the proportion of trajectories with errors exceeding a predefined threshold, penalizing plans with poor spatial alignment.

Nonreactive Closed-Loop Score (NR-CLS) evaluates trajectory execution within a simulated environment, where the ego vehicle operates in a static world without reactive background agents. This score reflects the planner’s ability to generate smooth, rule-compliant trajectories while maintaining lane discipline and avoiding illegal maneuvers.

Reactive Closed-Loop Score (R-CLS) extends the NR-CLS evaluation by incorporating simulated traffic participants governed by the Intelligent Driver Model (IDM). This setting introduces interactive complexity, requiring the planner to respond appropriately to leading or crossing vehicles, pedestrians, and other dynamic actors. R-CLS therefore reflects both motion feasibility and interactive robustness.

All three scores range from 0 to 100, with higher values indicating better planning performance. Together, they offer a comprehensive assessment of trajectory prediction quality, interpretability, and real-world deployability.

4.4 Main Results

We evaluated our method, **Align2Act**, on two benchmark splits from the nuPlan dataset: *Test14-random* and *Test14-hard*. These benchmarks include diverse and challenging scenarios representative of both common urban driving tasks and edge cases. Our planner was deployed within the nuPlan simulation environment, using an LQR (Linear Quadratic Regulator) controller for trajectory execution, and a kinematic vehicle model to update state transitions.

As shown in Table 1, Align2Act achieved strong results across all metrics. On *Test14-random*, we obtained an **OLS of 85.17**, with NR-CLS and R-CLS scores of **70.31** and **66.96**, respectively. On the

more challenging *Test14-hard* set, Align2Act achieved an OLS of **81.12**, demonstrating the model’s generalization to difficult edge cases. Compared to rule-based and hybrid planners such as IDM and PDM, our approach consistently outperformed in open-loop settings and remained competitive in closed-loop evaluations.

Table 1: Comparison of Align2Act variants with baseline planners on the nuPlan benchmark. Metrics are OLS, NR-CLS, and R-CLS on Test14-random and Test14-hard splits.

Planner	Test14-random			Test14-hard		
	OLS	NR-CLS	R-CLS	OLS	NR-CLS	R-CLS
Expert (LogReplay)	100.0	94.03	75.86	100.0	85.96	68.80
Rule-based						
IDM [28]	34.15	70.39	72.42	20.07	56.16	62.26
PDM-Closed [21]	46.32	90.05	91.64	26.43	65.07	75.18
Hybrid						
GameFormer [29]	79.35	80.80	79.31	75.27	66.59	68.83
PDM-Hybrid [21]	82.21	90.20	91.56	73.81	65.95	75.79
Learning-based						
PlanCNN [30]	62.93	69.66	67.54	52.40	49.47	52.16
UrbanDriver [19]	82.44	63.27	61.02	76.90	51.54	49.07
GC-PGP [31]	77.33	55.99	51.39	73.78	43.22	39.63
PDM-Open [21]	84.14	52.80	57.23	79.06	33.51	35.83
PlanTF [27]	<u>87.07</u>	<u>86.48</u>	<u>80.59</u>	<u>83.32</u>	<u>72.68</u>	<u>61.70</u>
LLM-based (Ours)						
Align2Act (LLaMA2 without chain)	61.23	46.91	42.47	68.02	55.14	50.31
Align2Act (InstructionChain)	<u>85.17</u>	70.31	66.96	<u>81.12</u>	57.37	52.95

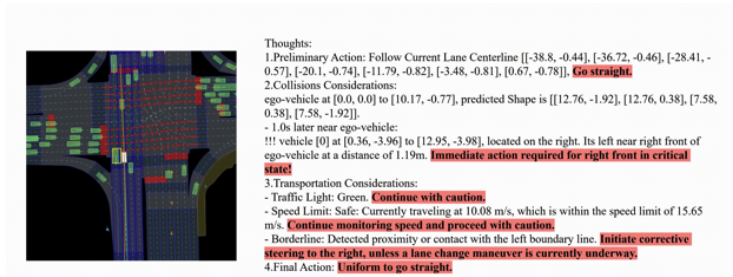


Figure 3: Visualization of Align2Act’s reasoning process for a lane-following scenario. The model’s decision-making chain is shown: (1) Preliminary action selection (blue trajectory), (2) Collision risk assessment (red bounding boxes for critical agents), (3) Traffic rule compliance (green light status and speed monitoring), and (4) Final action execution (bold trajectory with corrective steering). The ego vehicle (yellow) maintains safe distances while adhering to lane boundaries.

4.5 Ablation and Analysis

We conducted a series of ablation studies to evaluate the impact of reasoning structure, scenario diversity, temporal context, and input representations on planning performance. Table 2.

Structured Reasoning. To measure the effect of reasoning, we removed the Align2ActChain module and evaluated a flat-input baseline. Across all metrics, performance declined. For example, in a 5-scenario test configuration, the Open-Loop Score (OLS) dropped from 67.69 (with reasoning) to 59.91 (without), and the Reactive Closed-Loop Score (R-CLS) fell from 41.25 to 37.35. These results highlight the importance of structured reasoning for accurate and interpretable planning.

Scenario Diversity. We further tested generalization by expanding the number of training scenarios from 5 to 7. This increased diversity improved performance even without the reasoning chain (OLS:

61.23), and led to stronger gains when combined with Align2ActChain (OLS: 69.10, R-CLS: 43.80), confirming that broader scenario exposure strengthens contextual understanding.

Temporal Correlation vs. Diversity. To assess the importance of temporality, we compared two models: one trained on 300,000 independent, non-temporal scenarios, and another trained on 20,000 sequences with 16 temporally correlated frames. The non-temporal model outperformed its temporal counterpart (OLS: 63.11 vs. 60.35, R-CLS: 24.83 vs. 22.10), suggesting that scenario diversity provides a stronger training signal than frame-level continuity under limited compute. These findings indicate that, when training budgets are constrained, maximizing environmental variety may offer better generalization than increasing temporal depth.

Previous Trajectory Input. We also tested using the planned trajectory from the previous frame as an additional input. This led to performance degradation across all metrics. We hypothesize that the model overfit to shallow mappings between consecutive outputs, neglecting environmental awareness and state reasoning.

Table 2: Ablation results showing the impact of reasoning, temporality, and scenario scale. Best scores in bold.

Model Variant	OLS	NR-CLS	R-CLS
No Reasoning Chain (5 scenarios)	59.91	38.45	37.35
No Reasoning Chain (7 scenarios)	61.23	39.18	38.27
With Align2ActChain (5 scenarios)	67.69	40.84	41.25
With Align2ActChain (7 scenarios)	69.10	42.90	43.80
Align2ActChain + 16 Temporal Frames + 20,000 scenes	60.35	22.90	22.10
Prev. Trajectory Input (PT, no temp.) + 300,000 scenes	63.11	25.52	24.83

5 Limitations

While Align2ActDriver improves interpretability through structured reasoning, it underperforms conventional learning-based and hybrid planners in closed-loop settings. The Align2ActChain enhances transparency but lacks the robustness needed for real-time deployment.

Despite using LoRA for efficient fine-tuning, reliance on large language models leads to high inference latency and memory demands, limiting feasibility for on-vehicle applications without further optimization.

The manually crafted instruction format also presents scalability challenges, particularly in unseen or highly diverse scenarios. Generalization may require adaptive prompting or automated instruction generation.

Evaluation was limited to a subset of nuPlan. Broader benchmarks such as val14 could reveal generalization limits. Moreover, the absence of visual or LiDAR inputs restricts reasoning in perceptually complex environments.

6 Conclusion

We introduced Align2ActDriver, a motion planning framework that uses instruction-tuned language models to produce interpretable, human-aligned trajectories. By modeling planning as language generation, it enables structured reasoning through the Align2ActChain.

Evaluations on nuPlan show competitive open-loop results and coherent, rule-following behavior. Though closed-loop performance lags behind traditional planners, Align2ActDriver offers strong interpretability and alignment with human driving logic.

Ablation studies highlight the benefits of temporal context, scenario diversity, and reasoning chains. Visualizations confirm the model’s capacity to respond to realistic driving cues.

Future work includes integrating visual inputs, reducing latency, and scaling to multi-agent and broader benchmarks like val14. This approach marks a step toward bridging generative reasoning with real-world autonomous control.

References

- [1] Jiahao Li, Zhiqi Li, and Tong Lu. Driving with internvl: Outstanding champion in the track on driving with language of the autonomous grand challenge at cvpr 2024. *arXiv preprint arXiv:2412.07247*, 2024.
- [2] Yang Dong, Hansheng Liang, Mingliang Zhai, Cheng Li, Meng Xia, Xinglin Liu, Mengjingcheng Mo, Jiaxu Leng, Ji Tao, and Xinbo Gao. Bevlm: Got-based integration of bev and llm for driving with language.
- [3] Chonghao Sima, Katrin Renz, Kashyap Chitta, Li Chen, Hanxue Zhang, Chengen Xie, Jens Beißwenger, Ping Luo, Andreas Geiger, and Hongyang Li. Drivelm: Driving with graph visual question answering. In *European Conference on Computer Vision*, pages 256–274. Springer, 2024.
- [4] Hao Shao, Yuxuan Hu, Letian Wang, Guanglu Song, Steven L Waslander, Yu Liu, and Hongsheng Li. Lmdrive: Closed-loop end-to-end driving with large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15120–15130, 2024.
- [5] Ruijun Zhang, Xianda Guo, Wenzhao Zheng, Chenming Zhang, Kurt Keutzer, and Long Chen. Instruct large language models to drive like humans. *arXiv preprint arXiv:2406.07296*, 2024.
- [6] Jinghan Peng, Jingwen Wang, Xing Yu, and Dehui Du. The system description of cps team for track on driving with language of cvpr 2024 autonomous grand challenge.
- [7] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [8] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [9] Hiren Thakkar and A Manimaran. Comprehensive examination of instruction-based language models: A comparative analysis of mistral-7b and llama-2-7b. In *2023 International Conference on Emerging Research in Computational Science (ICERCS)*, pages 1–6. IEEE, 2023.
- [10] Kangtong Mo, Wenyan Liu, Xuanzhen Xu, Chang Yu, Yuelin Zou, and Fangqing Xia. Fine-tuning gemma-7b for enhanced sentiment analysis of financial news headlines. In *2024 IEEE 4th International Conference on Electronic Technology, Communication and Information (ICETCI)*, pages 130–135. IEEE, 2024.
- [11] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [12] Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, et al. Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. *arXiv preprint arXiv:2305.17144*, 2023.
- [13] Joon Sung Park, Joseph C O’Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- [14] Hongxin Zhang, Weihua Du, Jiaming Shan, Qinhong Zhou, Yilun Du, Joshua B Tenenbaum, Tianmin Shu, and Chuang Gan. Building cooperative embodied agents modularly with large language models. *arXiv preprint arXiv:2307.02485*, 2023.
- [15] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

- [16] Martin Treiber, Ansgar Hennecke, and Dirk Helbing. Congested traffic states in empirical observations and microscopic simulations. *Physical Review E*, 62(2):1805, 2000.
- [17] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Praseen Goyal, Lawrence D Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.
- [18] Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4693–4700. IEEE, 2018.
- [19] Oliver Scheel, Luca Bergamini, Maciej Wolczyk, Błażej Osipiński, and Peter Ondruska. Urban-driver: Learning to drive from real-world demonstrations using policy gradients. In *Conference on Robot Learning*, pages 718–728. PMLR, 2022.
- [20] Jie Cheng, Yingbing Chen, Xiaodong Mei, Bowen Yang, Bo Li, and Ming Liu. Rethinking imitation-based planner for autonomous driving. *arXiv preprint arXiv:2309.10443*, 2023.
- [21] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning. *arXiv preprint arXiv:2306.07962*, 2023.
- [22] Hao Sha, Yao Mu, Yuxuan Jiang, Li Chen, Chenfeng Xu, Ping Luo, Shengbo Eben Li, Masayoshi Tomizuka, Wei Zhan, and Mingyu Ding. LanguageMPC: Large language models as decision makers for autonomous driving. *arXiv preprint arXiv:2310.03026*, 2023.
- [23] Jiageng Mao, Yuxi Qian, Hang Zhao, and Yue Wang. Gpt-driver: Learning to drive with gpt. *arXiv preprint arXiv:2310.01415*, 2023.
- [24] Long Chen, Oleg Sinavski, Jan Hünemann, Alice Karnsund, Andrew James Willmott, Danny Birch, Daniel Maund, and Jamie Shotton. Driving with llms: Fusing object-level vector modality for explainable autonomous driving. *arXiv preprint arXiv:2310.01957*, 2023.
- [25] SP Sharan, Francesco Pittaluga, Manmohan Chandraker, et al. Llm-assist: Enhancing closed-loop planning with language-based reasoning. *arXiv preprint arXiv:2401.00125*, 2023.
- [26] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- [27] Jie Cheng, Yingbing Chen, Xiaodong Mei, Bowen Yang, Bo Li, and Ming Liu. Rethinking imitation-based planners for autonomous driving, 2023.
- [28] Zhiqi Li, Zhiding Yu, Shiyi Lan, Jiahao Li, Jan Kautz, Tong Lu, and Jose M Alvarez. Is ego status all you need for open-loop end-to-end autonomous driving? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14864–14873, 2024.
- [29] Zhiyu Huang, Haochen Liu, and Chen Lv. Gameformer: Game-theoretic modeling and learning of transformer-based interactive prediction and planning for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3903–3913, 2023.
- [30] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021.
- [31] Marcel Hallgarten, Martin Stoll, and Andreas Zell. From prediction to planning with goal conditioned lane graph traversals. In *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, pages 951–958. IEEE, 2023.